

Indian Institute Of Information Technology
Vadodara.

CampusEats — Complete Project Documentation

CS543: Web Services Project

Project Name: CampusEats
Course: CS543 — Web Services
Team: Team 10

Submitted By

1. Brajakshi Mandloi — Roll No. 20252651016
2. Vaishnavi Tiwari — Roll No. 20252651061

GitHub Repository: https://github.com/brajakshi1427-alt/campuseats

Project Demonstration Video: To be added after uploading the final demonstration video to Google Drive.

---

1. Introduction

CampusEats is a campus food-ordering web service designed to help students and other campus users discover food items, place orders, make payments, and track order status.

The project applies web service concepts to a practical food-ordering system. It uses a service-oriented architecture in which different services are responsible for specific business functions. Each service manages its own data and communicates with other services through defined interfaces and contracts.

The main objective of CampusEats is to demonstrate how a complex application can be divided into smaller, independent services. This structure improves maintainability, supports independent development, and reduces unnecessary dependencies between system components.

The project also covers HTTP communication, API design, service contracts, relational database design, external SOAP partner integration, and REST API reliability concepts.

2. Project Objectives

The primary objectives of CampusEats are:

- To provide a catalogue of available campus food items.
- To allow users to search and browse food items.
- To enable users to create and place food orders.
- To validate food-item availability before order confirmation.
- To integrate payment processing into the ordering workflow.
- To allow users to track order and payment status.
- To divide the application into services with clear responsibilities.
- To define service interfaces and communication contracts.
- To design a relational database schema for application data.
- To investigate HTTP requests and responses using browser developer tools.
- To integrate an external SOAP payment partner through a SOAP message contract.

- To apply web service reliability, validation, security, and testing practices.

3. Scope of the Project

CampusEats focuses on the design and implementation of a web-based campus food-ordering system.

The project scope includes:

1. Project requirements and system brief.
2. HTTP request and response investigation.
3. Service-oriented system design.
4. Service contracts and business operation specifications.
5. Relational database schema design.
6. REST API behavior and error handling.
7. External SOAP payment partner integration.
8. API reliability and performance considerations.
9. Documentation, testing, and GitHub-based submission.

The CampusPay partner details used in the SOAP assignment are illustrative. The example endpoint and credentials are placeholders and must not be treated as a real production payment gateway.

4. Main Users and Stakeholders

4.1 Campus Users and Students

Campus users interact with the system to:

- Browse the food catalogue.
- Search for food items.
- Select items and prepare an order.
- Place an order.
- Make or initiate a payment.
- Check order status.
- View relevant order information.

4.2 Campus Food and Catalogue Staff

Catalogue staff are responsible for maintaining food catalogue information, including:

- Food-item names and descriptions.
- Categories.
- Prices.

- Item availability.

4.3 Order Service

The Order Service manages the ordering process, including order creation, order items, order status, and coordination with other services.

4.4 Payment Service

The Payment Service manages payment-related information and coordinates payment requests with the payment partner.

5. Main System Capabilities

CampusEats is designed around the following capabilities.

Capability| Description
Browse Catalogue| Display available food items.
Search Catalogue| Help users locate food items.
Manage Cart and Order| Organize selected items before ordering.
Place Order| Create an order after required validations.
Process Payment| Coordinate payment processing.
Manage User Account| Maintain user-related information.
Check Item Availability| Verify whether selected food items are available.
Track Order Status| Provide the current state of an order.

These capabilities are supported by separate services with defined responsibilities.

6. Assignment 1 — HTTP Investigation and System Brief

6.1 Objective

The objective of Assignment 1 is to understand the CampusEats problem domain and investigate how web applications communicate using HTTP.

This assignment establishes the initial requirements and documents the observed HTTP request-response behavior.

6.2 System Brief

CampusEats is intended to help campus users discover food and place orders through a web-based system. The system must coordinate catalogue information, user details, order creation, availability checks, and payment processing.

The application is divided into logical services so that each business responsibility can be managed independently.

6.3 HTTP Investigation

HTTP is the communication protocol commonly used by web applications and REST APIs. A client sends an HTTP request to a server, and the server returns an HTTP response.

An HTTP request can contain:

- Request method.
- URL or endpoint.
- Headers.
- Optional request body.

An HTTP response can contain:

- Status code.
- Response headers.
- Optional response body.

Common HTTP methods include:

Method| Purpose
GET| Retrieve a resource.
POST| Create a resource or initiate an operation.
PUT| Replace or update a resource representation.
PATCH| Partially update a resource.
DELETE| Remove a resource.

6.4 HTTP Status Codes

Some important HTTP status codes relevant to CampusEats are:

Status Code| Meaning
200 OK| The request was successful.
201 Created| A resource was created successfully.
204 No Content| The request succeeded without a response body.
400 Bad Request| The request contains invalid data.
401 Unauthorized| Authentication is required or invalid.
403 Forbidden| The client is not permitted to perform the operation.
404 Not Found| The requested resource does not exist.
409 Conflict| The request conflicts with the current resource state.
429 Too Many Requests| The client has exceeded a rate limit.

500 Internal Server Error| An unexpected server-side error occurred.
502 Bad Gateway| A gateway received an invalid response from an upstream service.
503 Service Unavailable| The service is temporarily unavailable.

The appropriate status code depends on the actual API contract and the result of the request.

6.5 Browser Developer Tools

Browser Developer Tools can be used to inspect network communication between a client and a web server.

The Network panel helps identify:

- Requested URLs.
- HTTP methods.
- Status codes.
- Request and response headers.
- Request payloads.
- Response data.
- Timing information.

The observations from the investigation are documented in "http-log.md" and "network-analysis.md".

6.6 Assignment 1 Deliverables

The repository includes the following Assignment 1 files:

- "README.md" — Project overview.
- "brief.md" — CampusEats system brief.
- "http-log.md" — HTTP request and response log.
- "network-analysis.md" — Browser Developer Tools Network analysis.

These documents establish the system context and record the HTTP investigation.

7. Assignment 2 — Service-Oriented Design and Database Schema

7.1 Objective

The objective of Assignment 2 is to design CampusEats as a service-oriented system. The design identifies business capabilities, service boundaries, data ownership, service interactions, contracts, and the database schema.

7.2 Service Design Principles

The system design follows these principles:

1. Single responsibility: Each service focuses on a particular business capability.
2. Data ownership: Each service owns the data associated with its responsibility.
3. Defined contracts: Services communicate through explicit operations and agreed request-response structures.
4. Loose coupling: A service should not depend unnecessarily on another service's internal implementation.
5. Independent development: Services can be developed and maintained separately when their contracts remain stable.
6. Validation: Requests must be checked before business operations are completed.

7.3 Identified Services

A. Catalogue Service

The Catalogue Service manages food catalogue information.

Data owned:

- "FoodItem"
- "Category"
- "Availability"

Responsibilities:

- Provide food-item information.
- Support catalogue browsing and searching.
- Maintain categories and prices.
- Check item availability.

B. Orders Service

The Orders Service manages orders and their lifecycle.

Data owned:

- "Order"
- "OrderItem"
- "OrderStatus"

Responsibilities:

- Create and manage orders.
- Store the items included in each order.
- Coordinate availability validation.
- Coordinate with the Users and Payments Services.
- Track order status.

C. Users Service

The Users Service manages user-related information.

Data owned:

- "User"
- "UserProfile"

Responsibilities:

- Maintain user information.
- Provide user details required by authorized operations.
- Support user-related validation.

D. Payments Service

The Payments Service manages payment-related information.

Data owned:

- "Payment"
- "PaymentStatus"

Responsibilities:

- Create payment records.
- Initiate payment processing.
- Communicate with the external payment partner.
- Record payment outcomes.
- Provide payment-status information.

7.4 Data Ownership

Each service is the authoritative owner of its own data.

Service| Owned Data
Catalogue Service| FoodItem, Category, Availability

Orders Service| Order, OrderItem, OrderStatus
Users Service| User, UserProfile
Payments Service| Payment, PaymentStatus

No two services share ownership of the same business data. Other services should request the required information through service interfaces instead of directly changing another service's data.

7.5 Cross-Service Communication

The system uses the following logical interactions:

- "Orders → Catalogue: checkItem(item)"
- "Orders → Users: getUser(user)"
- "Orders → Payments: createPayment(order)"
- "Orders → Payments: getPaymentStatus(order)"

These interactions allow the Orders Service to coordinate the ordering workflow without taking ownership of catalogue, user, or payment data.

7.6 Central Business Operation: placeOrder

The "placeOrder" operation represents the central business workflow.

A typical logical sequence is:

1. The client submits an order request.
2. The system validates the request structure.
3. The Orders Service checks that the user is valid.
4. The Orders Service requests availability validation from the Catalogue Service.
5. The system calculates or verifies the order total according to the agreed contract.
6. The order is created according to the application's order-creation rules.
7. The Payments Service is asked to initiate payment processing.
8. The payment result is recorded or retrieved.
9. The order status is updated according to the payment and order-processing rules.
10. The client receives an appropriate response.

The exact transaction boundaries, rollback behavior, and status transitions must follow the implemented service contracts. A distributed workflow should not assume that changes in multiple services occur as one automatic database transaction.

7.7 Database Schema

The relational schema represents the principal entities required by CampusEats.

The main entities are:

- Users
- User profiles
- Food items
- Categories
- Item availability
- Orders
- Order items
- Payments
- Order and payment statuses

A relational design should define primary keys, foreign keys, appropriate data types, and constraints. Foreign keys are used where relationships are managed within the same database schema. In a distributed service architecture, cross-service relationships may instead be represented using identifiers and validated through service contracts.

The SQL definitions are provided in "schema.sql".

7.8 Assignment 2 Deliverables

The repository includes:

- "design.pdf" — System capabilities, service design, team information, contracts, "placeOrder" specification, and service validation.
- "services.drawio" — Editable service-design diagram.
- "services.png" — Exported service-design diagram.
- "schema.drawio" — Editable database ER diagram.
- "schema.png" — Exported database ER diagram.
- "schema.sql" — Database table definitions.

8. Assignment 3 — Integrating an External SOAP Partner

8.1 Objective

The objective of Assignment 3 is to integrate an external SOAP-based payment partner into CampusEats.

SOAP is a messaging protocol that commonly uses XML envelopes to exchange structured messages. The integration demonstrates how the Payments Service can send a payment request, process a SOAP response, and handle SOAP faults returned by a partner.

8.2 Integration Overview

The hypothetical external partner is named CampusPay.

The Payments Service acts as the CampusEats-side integration component. It constructs the SOAP request, sends it to the configured partner endpoint, interprets the response, and converts partner-specific outcomes into application-level payment statuses.

Important: The endpoint, WSDL address, merchant identifier, API key, order identifier, and card token in the examples are demonstration values. The ".example.com" domain is reserved for illustrative examples. These values do not provide access to a real payment service.

8.3 Service Discovery

The integration design identifies the partner through its Web Services Description Language (WSDL) document.

The illustrative WSDL location is:

"https://api.campus-pay.example.com/wsdl/payment.wsdl"

A WSDL document can describe:

- Available service operations.
- Input and output message structures.
- XML data types.
- Service bindings.
- The endpoint used by a client.

In a real integration, the developer must obtain the partner's official WSDL and confirm its supported operations and endpoint.

8.4 SOAP Operation

The example operation is:

"chargePayment"

Illustrative endpoint:

"https://api.campus-pay.example.com/soap/payment"

HTTP method:

"POST"

Content-Type:

"text/xml; charset=utf-8"

Illustrative SOAPAction:

"http://campus-pay.example.com/payment/chargePayment"

The exact endpoint, SOAP version, SOAPAction, namespaces, and XML structure must match the actual partner contract.

8.5 Example Payment Request

The following values illustrate the fields that may be included in a payment request:

- Merchant ID: "CE-MERCHANT-001"
- API key: "DEMO-KEY-12345"
- Order ID: "ORD-1001"
- Amount: "500.00"
- Currency: "INR"
- Card token: "TOKEN-987654"

These are placeholders for documentation and testing. They are not real credentials or payment details.

A SOAP request generally contains an Envelope, a Body, and an operation-specific XML element. Authentication details and payment fields must follow the actual partner's security and message contract.

8.6 Successful Payment Response

A successful partner response may contain:

- A transaction identifier.
- A payment status.
- A response message.

Illustrative response values:

- Transaction ID: "TXN-20260821-1001"
- Status: "SUCCESS"
- Message: "Payment successful"

The Payments Service should validate the response structure and map the partner's result to the application's payment model. It should not mark a payment as successful merely because an HTTP request was sent successfully.

8.7 SOAP Fault Handling

A SOAP partner may return a SOAP Fault if a payment cannot be completed.

The example fault describes the following situation:

- Fault message: "Payment could not be completed"
- Partner error code: "card_declined"
- Description: "The payment instrument was declined by the issuer."

The Payments Service should parse the fault safely and map the partner-specific error to an application-level error.

Partner Error| CampusEats Error| User-Facing Message
"card_declined"| "PAYMENT_DECLINED"| Payment declined. Please try another payment method.

The internal response may contain diagnostic details useful for developers, but the client should receive a clear and safe error message. Sensitive credentials and payment information must not be included in logs or user-facing errors.

8.8 SOAP Integration Workflow

1. The Orders Service requests payment processing.
2. The Payments Service validates the payment request.
3. The Payments Service creates the partner-specific SOAP XML envelope.
4. The request is sent to the configured partner endpoint.
5. The partner returns a SOAP response or SOAP Fault.
6. The Payments Service validates and parses the response.
7. A successful payment is mapped to the appropriate internal status.
8. A partner fault is mapped to a meaningful application error.
9. The payment outcome is stored according to the implementation.
10. The relevant payment status is made available to the ordering workflow.

8.9 Error Handling and Reliability

The integration should account for:

- Network timeouts.
- Connection failures.

- Invalid XML responses.
- Unexpected SOAP faults.
- Partner-side service failures.
- Duplicate payment requests.
- Unknown or unrecognized partner error codes.

Payment requests require particular care because blindly retrying a timed-out request can cause duplicate charges if the partner processed the original request. Where supported, a stable idempotency or transaction reference should be used, and the final payment state should be checked before retrying.

8.10 Assignment 3 Deliverables

The SOAP assignment resources include the following example files, where present in the repository:

- "integration.pdf" — Integration explanation.
- "partner.wsdl" — Partner service description.
- "soap-request.xml" — Example SOAP request.
- "soap-response.xml" — Example successful response.
- "soap-fault.xml" — Example SOAP Fault.

The examples document the intended integration contract. A successful live transaction cannot be claimed unless it has actually been tested against a real, authorized payment partner.

9. Assignments 4–6 — REST API Behavior, Reliability, and Quality

The later project work covers REST/API implementation and web service quality concerns. The following sections document the relevant API concepts used in the CampusEats project. The exact allocation of each feature to an individual assignment should follow the corresponding official assignment brief and the actual repository implementation.

9.1 REST API Design

REST APIs expose resources through URLs and use HTTP methods to perform operations.

A well-designed CampusEats API should:

- Use resource-oriented endpoints.
- Apply suitable HTTP methods.
- Validate incoming requests.
- Return meaningful status codes.
- Use consistent response formats.
- Document request and response structures.

- Handle invalid input and missing resources predictably.

Illustrative resource names could include catalogue items, orders, users, and payments. Actual endpoint paths must be taken from the implemented API rather than assumed from these examples.

9.2 JSON Request and Response Handling

JSON is commonly used to exchange structured data between REST clients and services.

API implementations should validate:

- Required fields.
- Data types.
- Numeric ranges.
- Valid identifiers.
- Allowed status values.
- Business rules such as item availability.

Invalid input should return a clear client error rather than an unexplained server exception.

Responses should use a consistent format so that clients can distinguish successful results from errors.

9.3 Idempotency

An operation is idempotent when repeating the same request under the defined conditions produces the same intended result as performing it once.

Idempotency is especially important for payment and order creation because clients may retry after a timeout or connection failure.

An idempotency mechanism can use a client-provided key associated with a stored result. The service should ensure that repeated requests using the same key do not unintentionally create duplicate orders or payments.

The precise implementation depends on the API contract and repository code.

9.4 ETags and Conditional Requests

An ETag is an HTTP response header that identifies a particular version of a resource representation.

Conditional requests can use headers such as:

ETag

If-None-Match

If-Match

These mechanisms can help reduce unnecessary data transfer or prevent updates based on stale resource versions.

For example, a client can retain an ETag and later request the same resource conditionally. If the representation has not changed, the server may return 304 Not Modified when the request and response semantics permit it.
ETags should only be documented as implemented when the corresponding API routes and tests actually support them.

9.5 Cross-Origin Resource Sharing (CORS)

CORS controls whether browsers allow web pages from one origin to access resources from another origin.
The server can configure allowed origins, methods, and headers. Configuration should be limited to the origins and operations that the application needs.
CORS is a browser access-control mechanism; it does not replace authentication, authorization, or server-side request validation.

9.6 Rate Limiting

Rate limiting restricts how frequently a client can call an API within a defined interval.
It helps reduce accidental overload and abuse. When a client exceeds a configured limit, the service can return HTTP 429 Too Many Requests.
Rate limits should be selected according to the application's expected traffic and operational requirements. The actual limit and enforcement mechanism must be documented from the implementation.

9.7 Gzip Compression

Gzip compression can reduce the size of suitable HTTP response bodies.
A client may advertise support through the Accept-Encoding header. When compression is supported and enabled, the server can return a compressed response and indicate the encoding using Content-Encoding.
Compression should be tested with representative responses. It is not necessary or beneficial for every response, especially very small payloads.

9.8 Consistent Error Responses

API errors should be understandable and consistent.
A useful error response may include:
An application error code.
A human-readable message.
Optional validation details.
A request or correlation identifier when supported.
Error responses should not expose passwords, API keys, payment tokens, stack traces, or other sensitive implementation details.

9.9 Testing

Testing should cover both successful and unsuccessful requests.
Relevant test cases include:
Valid request with a successful response.
Missing required fields.
Invalid data types.
Invalid or unknown resource identifiers.
Unavailable catalogue items.
Payment rejection.

Partner timeout or malformed response.
Duplicate request handling where idempotency is implemented.
Conditional requests where ETags are implemented.
CORS behavior where applicable.
Rate-limit behavior where implemented.
Compressed response behavior where enabled.
Automated tests should be run against the current version of the repository before reporting their results. Any stated test count or pass rate must reflect the actual test output.

10. System Architecture

CampusEats follows a service-oriented architecture.
At a logical level, the application consists of a client, the Orders Service, the Catalogue Service, the Users Service, the Payments Service, and an external payment partner.
The client initiates user-facing operations. The Orders Service coordinates the order workflow and communicates with other services through their defined interfaces. The Payments Service isolates payment-related responsibilities and the SOAP partner integration.
The architecture separates business responsibilities and establishes clear ownership of data. This makes it easier to change a service implementation without unnecessarily changing the entire system, provided that service contracts remain compatible.

11. Data Management

Data management is based on service ownership and relational modelling.
The database schema should define:
Primary keys for entity identification.
Foreign keys where relationships are enforced within the same database.
Appropriate data types.
Required-field constraints.
Uniqueness constraints where necessary.
Valid relationships between orders and order items.
Appropriate references between payment records and orders.
Order and payment statuses should follow defined state transitions. The system should avoid assuming that an order is completed merely because it was created or that a payment succeeded merely because the partner endpoint responded.

12. Security Considerations

Security is important because CampusEats handles user, order, and payment-related information.
Recommended practices include:
Validate all incoming requests on the server.
Authenticate and authorize protected operations.
Store secrets in environment variables or a secure secret-management system.
Never commit real API keys, passwords, or payment credentials to GitHub.
Use HTTPS for real network communication.
Avoid logging card details, API keys, or payment tokens.
Return safe and understandable error messages.
Apply rate limits to sensitive endpoints where appropriate.
Validate responses received from external services.

Use timeouts and controlled error handling for partner calls.
Avoid treating illustrative credentials as production credentials.
Verify payment results through the defined partner contract.
For a real payment deployment, use the payment provider's approved tokenization and security requirements. Do not store raw card information unless the system is explicitly designed and authorized to meet the applicable security requirements.

13. Tools and Technologies

The project uses or documents the following tools and concepts:

Tool or Technology	Purpose
HTTP and REST	Web communication and resource-based APIs.
SOAP and XML	External payment partner messaging.
WSDL	Description of SOAP operations and message contracts.
Browser Developer Tools	Inspection of HTTP requests and responses.
curl	Manual HTTP/API request testing.
Git	Version control and incremental changes.
GitHub	Source-code hosting and project submission.
Draw.io	Editable service and database diagrams.
SQL	Relational database schema definition.
Automated tests	Verification of expected API and integration behavior.

The exact runtime, programming language, database engine, and test framework should be identified from the actual project files.

14. Repository Structure

The main repository documentation and design resources are organized as follows:

```
campuseats/
├── README.md
├── brief.md
├── http-log.md
├── network-analysis.md
├── design.pdf
├── services.drawio
├── services.png
├── schema.drawio
├── schema.png
├── schema.sql
└── docs/
    └── project-documentation.md
```

The SOAP integration resources and implementation files should also be included in the repository in their appropriate locations if they are part of the final submission.
Each file should contain the actual corresponding content. Listing a filename in the documentation does not replace uploading the file itself.

15. Git and GitHub Workflow

Git is used to track changes throughout the project. GitHub provides the shared repository for project files, implementation, and documentation.

A recommended workflow is:
Create or clone the project repository.
Add the system brief and project overview.
Commit HTTP investigation documents.
Add service diagrams and database design.
Add service contracts and implementation files.
Add SOAP request, response, and fault examples.
Add tests and supporting documentation.
Review the final repository structure.
Verify that sensitive credentials and generated temporary files are not committed.
Push the final changes to GitHub.
Meaningful commit messages make it easier to understand how the project evolved.
16. Validation and Final Review
Before submission, review the following items:
The README explains the project and links to the relevant documentation.
The system brief reflects the CampusEats requirements.
The HTTP log and network analysis are included.
Service responsibilities and data ownership are documented.
The service diagram and database ER diagram are available.
The SQL schema is included.
Service contracts and the placeOrder workflow are documented.
SOAP request, successful response, and fault examples are available where required.
Partner-specific errors are mapped to understandable application errors.
API behavior and reliability features are described consistently with the actual code.
Automated tests are executed and their results are recorded accurately.
The GitHub repository is accessible to the intended reviewers.
The final demonstration video is uploaded to Google Drive.
The Google Drive link is configured as Anyone with the link can view, if permitted by the course requirements.
The demonstration link is added to the README and submission report.
The final submission follows the instructor's instructions.
17. Team Contributions
Brajakshi Mandloi — Roll No. 20252651016
Contribution areas include project documentation, service-oriented design, web-service analysis, integration documentation, and preparation of submission materials, according to the work completed by the team.
Vaishnavi Tiwari — Roll No. 20252651061
Contribution areas include system design, database and service documentation, API and integration work, testing, and preparation of submission materials, according to the work completed by the team.
The final submission should reflect the actual contribution of each team member.
18. Limitations and Future Improvements
Possible limitations and future improvements include:

Replacing illustrative SOAP partner details with an authorized, working payment-provider integration.
Improving authentication and authorization.
Adding stronger monitoring and structured logging.
Expanding automated integration and failure-path tests.
Improving resilience against network failures.
Defining more detailed order and payment state transitions.
Improving API documentation and validation.
Adding operational monitoring and performance testing.
Refining user experience and accessibility.
Deploying the services to a suitable production-like environment.
These are potential improvements and should not be interpreted as features already implemented.

19. Conclusion

CampusEats demonstrates how web-service principles can be applied to a campus food-ordering system. The project defines separate services for catalogue management, orders, users, and payments, with explicit responsibilities and data ownership.

The HTTP investigation provides an understanding of request-response communication. The service-oriented design establishes clear boundaries and cross-service interactions. The relational schema documents the main data entities, while the SOAP integration design demonstrates how an external payment partner can be called and how success and failure responses can be handled.

The later API work addresses request validation, response consistency, reliability, security, and testing. Together, these areas provide a structured foundation for developing a maintainable and extensible web-service application.

The project also demonstrates the importance of clear documentation, version control, testing, and responsible handling of external service credentials.

GitHub Repository: https://github.com/brajakshi1427-alt/campuseats

Demonstration Video: Add the final Google Drive link after uploading the completed demonstration.

End of Documentation.